

LAB 4: VIBE CODING — PROMPT & CHAT LOG

Student: A R Akshay Kumar | USN: PES1UG24CS705 | Assigned Repo: #41 (SETAPESU26/41_fruit-ninja)

Prompt 1: Initial Diagnosis & Fixing the Broken Codebase

Prompt: User: 'I have been assigned repository #41 (SETAPESU26/41_fruit-ninja). The code provided is broken and will not run. Please inspect main.py, fruit.py, and game_engine.py, identify why it fails, and provide fixes.'

Solution: Resolution: Identified incomplete syntax in main.py ('while ru'), fruit.py (contains_point), and game_engine.py (handle_event). Completed event loop, Euclidean distance collision, and mouse trail tracking to establish a functional 60 FPS Pygame baseline.

Prompt 2: Task 1 — Refine Continuous Collision Detection

Prompt: User: 'Task 1: Fast swipes pass right through fruit without slicing it. Investigate and enhance slice detection so quick swipes register reliably.'

Solution: Resolution: Diagnosed tunneling caused by discrete frame sampling. Formulated continuous segment-to-point Euclidean projection in Fruit.intersects_segment(p1, p2). Clamps projection parameter t to [0,1]. Fast swipes register 100% reliably regardless of velocity.

Prompt 3: Task 2 & Task 3 — Game Over Screen & Replay Difficulty

Prompt: User: 'Implement Task 2 (Game Over modal with final score) and Task 3 (Replay option with Easy, Medium, Hard difficulty, or exit).'

Solution: Resolution: Built interactive Game Over modal displaying defeat reason, final score, and best score. Implemented Easy (interval 70, 8% bombs), Medium (interval 52, 16% bombs), and Hard (interval 36, 28% bombs) selectable via mouse clicks or hotkeys [1, 2, 3, Q].

Prompt 4: Task 4 — Procedural Sound Feedback

Prompt: User: 'Task 4 requires sound effects for slicing fruit, hitting bomb, and game-over moment without external MP3/WAV downloads. Can we synthesize them in Python?'

Solution: Resolution: Created game/audio.py using standard wave and struct modules to generate 16-bit 44.1 kHz PCM audio files: slice.wav (swept whoosh), bomb.wav (explosive rumble), and game_over.wav (descending arpeggio). 100% self-contained.